

HDOC Day-18

NAME: Yug Nirwal

SAP ID: 590033354

SUBJECT: C programming

DATE: 11-09-26

Problem Statement

Write a menu-driven C program using switch-case to perform digit-based operations on a positive integer. Use one separate user-defined function for each menu option. Every function must have a void return type, take no arguments, read its own input, process it, and display the result.

- Check whether the number is a Strong number.
- Check whether the number is an Armstrong number.
- Check whether the number is a Harshad number.
- Check whether the number is a Duck number.

1. Algorithm

1. Start the program.
2. Display a menu with four choices: Strong, Armstrong, Harshad, and Duck number.
3. Read the user's menu choice.
4. Use switch-case to call the corresponding void function. No value is passed to the function.
5. Inside strongNumber(), read a positive integer, find the factorial of each digit, add the factorials, and compare the sum with the original number.
6. Inside armstrongNumber(), read a positive integer, count its digits, add each digit raised to that digit count, and compare the sum with the original number.
7. Inside harshadNumber(), read a positive integer, find the sum of its digits, and check whether the original number is exactly divisible by that sum.
8. Inside duckNumber(), read the number as a string and check whether any digit after the first digit is 0.
9. Display the result inside the selected function. For an invalid menu choice, display an error message.
10. Stop the program.

2. Test Case Table

TC	Operation	Input	Expected Result	Actual Result	Status
1	Strong	145	Strong number	Strong number	Pass
2	Strong	123	Not a Strong number	Not a Strong number	Pass
3	Armstrong	153	Armstrong number	Armstrong number	Pass
4	Armstrong	125	Not an Armstrong number	Not an Armstrong number	Pass
5	Harshad	18	Harshad number	Harshad number	Pass
6	Harshad	19	Not a Harshad number	Not a Harshad number	Pass
7	Duck	1023	Duck number	Duck number	Pass

TC	Operation	Input	Expected Result	Actual Result	Status
8	Duck	1234	Not a Duck number	Not a Duck number	Pass
9	Menu	7	Invalid choice	Invalid choice	Pass

3. C Program

```
#include <stdio.h>
#include <string.h>

void strongNumber(void);
void armstrongNumber(void);
void harshadNumber(void);
void duckNumber(void);

int main(void)
{
    int choice;

    printf("DIGIT OPERATIONS MENU\n");
    printf("1. Check Strong Number\n");
    printf("2. Check Armstrong Number\n");
    printf("3. Check Harshad Number\n");
    printf("4. Check Duck Number\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            strongNumber();
            break;
        case 2:
            armstrongNumber();
            break;
        case 3:
            harshadNumber();
            break;
        case 4:
            duckNumber();
            break;
        default:
            printf("Invalid choice\n");
    }

    return 0;
}

void strongNumber(void)
{
    int n, original, temp, digit, i, factorial, sum = 0;

    printf("Enter a positive integer: ");
    scanf("%d", &n);
    original = n;
    temp = n;

    while (temp > 0)
    {
        digit = temp % 10;
        factorial = 1;
        for (i = 1; i <= digit; i++)
            factorial = factorial * i;
        sum = sum + factorial;
    }
}
```

```

        temp = temp / 10;
    }

    if (sum == original)
        printf("%d is a Strong number.\n", original);
    else
        printf("%d is not a Strong number.\n", original);
}

void armstrongNumber(void)
{
    int n, original, temp, digit, digits = 0, i;
    int power, sum = 0;

    printf("Enter a positive integer: ");
    scanf("%d", &n);
    original = n;
    temp = n;

    while (temp > 0)
    {
        digits++;
        temp = temp / 10;
    }

    temp = n;
    while (temp > 0)
    {
        digit = temp % 10;
        power = 1;
        for (i = 0; i < digits; i++)
            power = power * digit;
        sum = sum + power;
        temp = temp / 10;
    }

    if (sum == original)
        printf("%d is an Armstrong number.\n", original);
    else
        printf("%d is not an Armstrong number.\n", original);
}

void harshadNumber(void)
{
    int n, original, temp, sum = 0;

    printf("Enter a positive integer: ");
    scanf("%d", &n);
    original = n;
    temp = n;

    while (temp > 0)
    {
        sum = sum + (temp % 10);
        temp = temp / 10;
    }

    if (sum != 0 && original % sum == 0)
        printf("%d is a Harshad number.\n", original);
    else
        printf("%d is not a Harshad number.\n", original);
}

void duckNumber(void)
{
    char number[100];
    int i, isDuck = 0;

```

```

printf("Enter a positive integer: ");
scanf("%99s", number);

for (i = 1; number[i] != '\0'; i++)
{
    if (number[i] == '0')
    {
        isDuck = 1;
        break;
    }
}

if (isDuck)
    printf("%s is a Duck number.\n", number);
else
    printf("%s is not a Duck number.\n", number);
}

```

4. Compilation

Save the program as hdoc_day18.c. In the terminal, compile it using:

```
gcc hdoc_day18.c -o hdoc_day18
```

The program compiles successfully without errors or warnings.

5. Execution

Run the compiled program using:

```
./hdoc_day18
```

The menu appears, the selected function asks for its own input, and the result is displayed on the screen.

6. Sample Execution and Testing

Test Case 1 - Strong Number (145)

```

(kali@kali) - [~/Desktop/cprog]
$ ./hdoc_day18
DIGIT OPERATIONS MENU
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 1
Enter a positive integer: 145
145 is a Strong number.

```

Test Case 2 - Strong Number (123)

```

(kali@kali) - [~/Desktop/cprog]
$ ./hdoc_day18
DIGIT OPERATIONS MENU
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 1
Enter a positive integer: 123
123 is not a Strong number.

```

Test Case 3 - Armstrong Number (153)

```
(kali@kali)-[~/Desktop/cprog]
$ ./hdoc_day18
DIGIT OPERATIONS MENU
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 2
Enter a positive integer: 153
153 is an Armstrong number.
```

Test Case 4 - Armstrong Number (125)

```
(kali@kali)-[~/Desktop/cprog]
$ ./hdoc_day18
DIGIT OPERATIONS MENU
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 2
Enter a positive integer: 125
125 is not an Armstrong number.
```

Test Case 5 - Harshad Number (18)

```
(kali@kali)-[~/Desktop/cprog]
$ ./hdoc_day18
DIGIT OPERATIONS MENU
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 3
Enter a positive integer: 18
18 is a Harshad number.
```

Test Case 6 - Harshad Number (19)

```
(kali@kali)-[~/Desktop/cprog]
$ ./hdoc_day18
DIGIT OPERATIONS MENU
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 3
Enter a positive integer: 19
19 is not a Harshad number.
```

Test Case 7 - Duck Number (1023)

```
(kali@kali) - [~/Desktop/cprog]
$ ./hdoc_day18
DIGIT OPERATIONS MENU
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 4
Enter a positive integer: 1023
1023 is a Duck number.
```

Test Case 8 - Duck Number (1234)

```
(kali@kali) - [~/Desktop/cprog]
$ ./hdoc_day18
DIGIT OPERATIONS MENU
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 4
Enter a positive integer: 1234
1234 is not a Duck number.
```

Test Case 9 - Invalid Menu Choice

```
(kali@kali) - [~/Desktop/cprog]
$ ./hdoc_day18
DIGIT OPERATIONS MENU
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 7
Invalid choice
```

7. Result

The menu-driven C program was written, compiled, executed, and tested successfully. It uses switch-case and four separate void user-defined functions with simple function calls. The program correctly identifies Strong, Armstrong, Harshad, and Duck numbers for the prepared test cases.